

VERI-PHY

INTELLIGENCE HUB

AUTH & CONNECTION — IMPLEMENTATION PROMPT

Role-based specification for developers implementing the Hub authentication system, the Squad API key connection flow, the native app bridge connection, and the seeded Super Admin account.

00 PROJECT CONTEXT & AUTH PHILOSOPHY

Project Context & Auth Philosophy

Veri-phy Intelligence Hub is a web-based data store and visualization layer for the Veri-phy ecosystem. It receives verified transaction data pushed by the native Android app via an active WebSocket connection. The Hub has its own authentication system — merchants create Hub accounts authenticated against their Squad credentials. The native app has its own separate authentication system — also Squad-authenticated. The connection between the Hub and the native app is a bridge key, generated by the Hub, entered into the native app. These are three distinct auth layers and must be implemented independently.

Core Principle: The Hub is the store. The native app is the conduit. Auth connects users to the store. The bridge key connects the conduit to the store. A merchant can use the native app without a Hub account — they just lose persistence and visualization. When the bridge is established, a live WebSocket carries verified data from native app to Hub in real time.

01 ROLE DEFINITIONS

Role Definitions

The system has four roles. Every database record, every API endpoint, and every UI element is scoped by role. Role is assigned at account creation and stored in the merchants/users table. Role escalation requires a Super Admin action.

Role	Identifier	Access Scope
------	------------	--------------

SUPER_ADMIN	<code>super_admin</code>	Full unrestricted access to all Hub data across all merchants. Can create/delete/suspend any account. Can view and query all verification records, receipts, anomalies, and audit logs system-wide. Can promote/demote any user role. Can access the seeded admin dashboard at /admin. This role is NEVER self-assigned — it is seeded only.
ADMIN	<code>admin</code>	Access to all merchant data across the platform. Read-only on financial records. Can manage merchant accounts (suspend, reset connection). Can view model health, system queues, and audit logs. Cannot delete records. Cannot access infrastructure settings. Assigned by SUPER_ADMIN only.
MERCHANT	<code>merchant</code>	Access to their own data only — enforced at query level by <code>merchant_id</code> scoping. Can view their verifications, receipts, anomalies, analytics. Can generate and manage their Hub bridge key. Can configure notification and webhook settings. Cannot see any other merchant's data under any circumstance.
SYSTEM	<code>system</code>	Internal machine role. Used by: Celery workers writing verification records, WebSocket server writing push events, Squad webhook receiver storing settlement data. Never exposed to UI. Never assigned to a human account. Identified by a service API key in request headers, not a JWT.

RBAC Implementation Note: Role is stored as an enum in the database. Every FastAPI endpoint is decorated with a role guard: `@require_role(["merchant", "admin"])`. Every database query for merchant-owned data appends `WHERE merchant_id = current_user.merchant_id` unless the caller is ADMIN or SUPER_ADMIN. There are no exceptions to this rule.

02 HUB AUTHENTICATION — SCREEN SPECIFICATION

Hub Authentication — Screen Specification

The Hub has a single auth entry point at /auth. It handles both new account creation and returning user login through the same interface. There is no separate /signup and /login — the system detects whether the Squad key belongs to an existing account or a new one and routes accordingly.

Screen: /auth — The Entry Gate

This is the only screen visible to unauthenticated users. All other routes redirect here via Next.js middleware. Design: full-screen dark background, centered card, minimal. Resend.com aesthetic. No decorations. The card contains:

Element	Specification
Logo + Wordmark	Veri-phy logo top-center of card. Small. Not the hero.
Headline	"Connect your Squad account" — 20px, not bold. One line. No tagline.
Squad Key Input	Single input field. Placeholder: sk_live_***** — DM Mono font. Type=password (masked by default). Eye icon to reveal. Full width of card. Auto-focus on mount.
Reveal/mask toggle	Eye icon right-side of input. Clicking toggles between masked and revealed. State tracked locally in component.
Continue button	Full width. Label: "Continue with Squad" — not "Login", not "Submit". Primary blue. Shows a spinner on submit. Disabled while loading.
Helper link	Below button: "Where do I find my Squad API key?" — opens a small popover explaining: Go to dashboard.squadco.com → Settings → API Keys → copy your Secret Key.
Error state	If Squad key is invalid: input border turns red, message below input: "We couldn't find a Squad account with this key. Check your key and try again." — not a toast, inline.
Loading state	Button shows spinner, input is disabled. Status text below button: "Verifying with Squad..." — this covers the network call to POST /api/v1/auth/squad/connect.

Auth Flow — Step by Step

Developer role: implement this exact sequence. No steps may be skipped or reordered.

```
STEP 1 — Merchant types Squad Secret Key into input field
  Key format: sk_live_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
  or          sk_test_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX (sandbox)

STEP 2 — Merchant clicks 'Continue with Squad'
  Frontend: POST /api/v1/auth/squad/connect
  Body: { squad_secret_key: 'sk_live_...' }
  Header: Content-Type: application/json
  NO auth header — this is the auth initiation endpoint
```

```

STEP 3 – Backend: validate_squad_key(key)
  Call Squad API: GET https://api.squadco.com/merchant/me
  Header: Authorization: Bearer {squad_secret_key}
  Timeout: 8 seconds

STEP 4a – Squad returns 200:
  Response contains: { merchant_id, business_name, email,
                      settlement_account, currency, ... }
  → Proceed to STEP 5

STEP 4b – Squad returns 401/403:
  → Return HTTP 401 to frontend
  → Frontend shows inline error: 'Invalid Squad key'
  → STOP. Do not create any record.

STEP 4c – Squad returns 5xx / timeout:
  → Return HTTP 503 to frontend
  → Frontend shows: 'Squad is unreachable. Try again.'
  → STOP. Do not create any record.

STEP 5 – Backend: upsert_merchant(squad_merchant_id)
  Check if merchants.squad_merchant_id already exists:

  IF EXISTS (returning user):
    Update: squad_key_encrypted = encrypt(key)
           squad_key_hint = mask(key) # 'sk_live_...3f2a'
           last_login_at = now()
    Role: unchanged – do NOT reset role on re-login

  IF NOT EXISTS (new account):
    Insert: id = uuid4()
           squad_merchant_id = from Squad response
           business_name = from Squad response
           email = from Squad response
           squad_key_encrypted = encrypt(key)
           squad_key_hint = mask(key)
           role = 'merchant' # default role always
           hub_connected = false
           created_at = now()

STEP 6 – Backend: issue_jwt(merchant_id, role)
  JWT payload: {
    sub: merchant.id, # Veri-phy UUID
    squad_mid: merchant.squad_merchant_id,
    role: merchant.role,
    iat: now(),
    exp: now() + 7 days
  }
  Signed with RS256 private key
  Return: { access_token, merchant: { id, business_name, role } }

STEP 7 – Frontend: store token via NextAuth session
  session.accessToken = access_token
  session.user.role = role
  session.user.merchantId = id
  Redirect to /overview

STEP 8 – Middleware on every subsequent request:
  Verify JWT signature
  Check exp – if expired, redirect to /auth
  Attach decoded payload to request context
  Role guard checks role against endpoint requirements

```

Session & Token Management

Property	Implementation
JWT Algorithm	RS256 — asymmetric. Private key signs (backend only). Public key verifies (middleware). Never HS256 — the secret would need to be in the frontend.
Access Token TTL	7 days. Long-lived because this is a dashboard app — merchants should not be re-prompted every session. Revocation handled via a token blacklist in Redis on explicit logout.
Storage	NextAuth httpOnly cookie — NOT localStorage. httpOnly means JavaScript cannot read it. Immune to XSS token theft.
Refresh	No refresh token. After 7 days, user is redirected to /auth and re-enters Squad key. This is intentional — re-validating the Squad key on re-auth ensures the key is still active.
Logout	DELETE /api/v1/auth/session — adds JWT jti to Redis blacklist (TTL = remaining JWT lifetime). Cookie cleared client-side. Redirect to /auth.
Middleware scope	All routes under / except /auth and /api/v1/auth/*. Implemented in middleware.ts at app root — runs on every request before any page renders.

03 HUB STATE AFTER AUTH — THE DISCONNECTED STATE

Hub State After Auth — The Disconnected State

When a merchant successfully authenticates for the first time, they land on /overview in a disconnected state. The Hub account exists. No native app has connected yet. The Hub is empty — no verifications, no receipts, no anomalies. This state must be handled gracefully, not shown as errors or broken UI.

Disconnected State — UI Rules

- ▶ The /overview page renders normally with all sections visible.
- ▶ Stat cards show: 0, 0, 0, 0 — with a subtle 'No data yet' label beneath each, not an error.
- ▶ Charts render empty axes with a centered message: 'Waiting for your first verification' — not a spinner, not an error state.
- ▶ The recent verifications table shows: 'No verifications yet. Connect your Veri-phy app to start.' — with a link that opens the connection modal.
- ▶ Every page in the Hub follows this pattern — empty state with a clear call to action pointing to the connection flow.

The Connection Indicator — Top Left of Sidebar

This component is the live status of the native app connection. It sits at the top of the sidebar, below the workspace switcher. It is always visible on every page. Its state drives the entire connection UX.

State	Visual + Behaviour
DISCONNECTED	Gray dot (6px, no animation). Label: 'No app connected'. Entire component is clickable. Cursor: pointer. Clicking opens the Connection Modal. Tooltip on hover: 'Connect your Veri-phy Android app'.
WAITING	Amber dot (6px, slow pulse animation 2s loop). Label: 'Waiting for connection...' — shown after key is generated and modal is still open. Non-clickable in this state.
CONNECTED	Green dot (6px, gentle breathe animation 3s loop). Label: merchant's device name e.g. 'Tecno Camon 20'. Clicking opens a small popover: device info + 'Disconnect' button. Tooltip: 'Live — data flowing'.
RECONNECTING	Amber dot (fast pulse 0.8s). Label: 'Reconnecting...' — shown when WebSocket drops and auto-reconnect is in progress. Non-clickable.
DISCONNECTED (was connected)	Gray dot. Label: 'App offline'. Clicking opens popover explaining the app went offline, with instructions to open the Android app. NOT the same as never-connected state — different messaging.

04 NATIVE APP CONNECTION — BRIDGE KEY FLOW

Native App Bridge Connection — Full Specification

The bridge key is a short-lived, single-use token generated by the Hub that the merchant enters into their Veri-phy Android app. It cryptographically links the native app's identity (authenticated via Squad key) to the Hub account (also authenticated via Squad key). Once linked, a persistent WebSocket carries data from app to Hub in real time.

The Connection Modal

Triggered when the merchant clicks the disconnected Connection Indicator in the sidebar. Rendered as a Radix UI Dialog over the current page — page is NOT navigated away. The modal has three internal states: generating, waiting, and connected.

Modal State 1 — Generating (< 300ms, almost instant)

```
On modal open:
Frontend calls: POST /api/v1/connection/generate-key
Header: Authorization: Bearer {merchant JWT}
Body: {}

Backend:
Generate token: vph_conn_{32 random bytes hex}
Example:        vph_conn_4f8a2d91b3e047c6a5d2f8b1e3094f7a
Insert into hub_connection_tokens:
  { token, merchant_id, created_at: now(), expires_at: now()+15min,
    used: false }
Return: { token, expires_at, qr_data }
qr_data = base64 encoded JSON:
  { t: token, v: '1', hub: 'app.veriphy.ng' }

Frontend renders the modal immediately with the key.
```

Modal State 2 — Waiting (up to 15 minutes)

The modal shows the generated key and waits for the native app to connect. The frontend polls the backend every 3 seconds to check if the key has been used.

```
MODAL UI — Waiting State:

Connect Veri-phy App ×
Enter this key in your Veri-phy Android app

vph_conn_4f8a2d91b3e047c6a5d2f8b1e3094f7a
[Copy Key] [Show QR Code]

Waiting for connection...
14:32 remaining
```

How to connect:

1. Open Veri-phy on your Android device
2. Tap Settings → Connect to Intelligence Hub
3. Paste the key above or scan the QR code

[Generate new key]

POLLING (runs every 3 seconds while modal is open):

GET /api/v1/connection/status?token={token}

Response options:

```
{ status: 'waiting' }    → keep polling
{ status: 'connected',
  device: { name, android_version, app_version } } → go to State 3
{ status: 'expired' }    → show expired state, offer new key
```

COUNTDOWN TIMER:

Computed client-side from expires_at returned by generate-key

Format: MM:SS (14:32 → 14:31 → ... → 00:00)

When reaches 00:00:

Stop polling

Show: 'This key has expired.'

Show button: 'Generate a new key'

Calling generate-key again invalidates the expired token

Modal State 3 — Connected

MODAL UI — Connected State:

Connect Veri-phy App ×

✓ Connected

Tecno Camon 20 · Android 13
Veri-phy v1.0.0 · Connected just now

Your app is now live. Verified transactions will appear in this Hub in real time.

[Done]

On Done:

Close modal

Connection Indicator updates to CONNECTED state (green dot)

WebSocket connection is now ACTIVE

Overview page re-fetches (TanStack Query invalidation)

Backend — Connection Establishment (what happens when native app sends the key)

Native app sends to backend:

POST /api/v1/connection/establish

Header: Authorization: Bearer {native app JWT}

Body: {

hub_connection_key: 'vph_conn_4f8a2d91b3e047c6a5d2f8b1e3094f7a',

device_info: {

name: 'Tecno Camon 20',

android_version: '13',

app_version: '1.0.0',

```

    device_id: '{Android ANDROID_ID}'
  }
}

Backend validation sequence:
1. Decode native app JWT → get native_merchant_id
2. Look up token in hub_connection_tokens WHERE token = key
3. Validate: token.exists? → else 404
4. Validate: token.used == false? → else 409 'Key already used'
5. Validate: token.expires_at > now()? → else 410 'Key expired'
6. Validate: token.merchant_id == native_merchant_id?
   → MUST match – same Squad account on both sides
   → else 403 'Key belongs to a different account'
7. Mark token as used: UPDATE hub_connection_tokens SET used=true
8. Upsert device record:
   hub_devices: { merchant_id, device_info, connected_at: now() }
9. Publish to Redis channel: hub:connection:{merchant_id}
   Payload: { status: 'connected', device: device_info }
   (This is what the Hub's polling endpoint reads)
10. Return 200: { status: 'connected', ws_url: 'wss://api.veriphy.ng/ws/live/{merchant_id}' }

After Step 10:
Native app opens WebSocket to ws_url
WebSocket authenticated with same JWT in header
Connection is live
Hub polling detects 'connected' in Redis → modal transitions to State 3

```

WebSocket — Live Data Channel

```

ENDPOINT: wss://api.veriphy.ng/ws/live/{merchant_id}
AUTH: JWT passed as query param: ?token={jwt}
      (WebSocket protocol does not support custom headers in browsers)

MESSAGE TYPES (native app → backend → Hub):

Type: verification_event
{
  type: 'verification_event',
  payload: {
    id: uuid,
    sms_sender: 'GTBank',
    amount: 24500.00,
    txn_ref: 'FTX2024...',
    verdict: 'VERIFIED' | 'PENDING' | 'FAKE',
    trust_score: 0.94,
    squad_result: 'CONFIRMED' | 'PENDING' | 'NO_RECORD',
    nlp_confidence: 0.97,
    source: 'sms',
    attribution: [{ signal: 'squad_ledger', weight: 0.52, direction: 'fake' }],
    created_at: ISO8601
  }
}

Type: receipt_event
{
  type: 'receipt_event',
  payload: {
    id: uuid,
    result: 'AUTHENTIC' | 'MANIPULATED' | 'UNVERIFIED',
    ela_score: 0.23,
    phash_match: false,
    amount_extracted: 15000.00,
    bank_identified: 'Access Bank',
  }
}

```

```
image_url: 'https://storage.veriphy.ng/receipts/{id}.jpg',
source: 'receipt',
created_at: ISO8601
}
}

Type: ping (every 30s to keep connection alive)
{ type: 'ping' }
Backend responds: { type: 'pong' }

On disconnect:
  Backend marks hub_devices.last_seen = now()
  Publishes to Redis: hub:presence:{merchant_id} = 'offline'
  Hub connection indicator transitions to DISCONNECTED (was connected) state
  Auto-reconnect from native app: exponential backoff 1s → 2s → 4s → 8s → max 30s
```

05 DATABASE SCHEMA — AUTH & CONNECTION TABLES

Database Schema — Auth & Connection Tables

```

-- PostgreSQL DDL

-- Core merchant identity
CREATE TABLE merchants (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  squad_merchant_id VARCHAR(100) UNIQUE NOT NULL,
  business_name     VARCHAR(255) NOT NULL,
  email            VARCHAR(255),
  squad_key_encrypted BYTEA NOT NULL,      -- AES-256-GCM encrypted
  squad_key_hint     VARCHAR(30),          -- 'sk_live_....3f2a' display only
  squad_account_number VARCHAR(20),        -- virtual account
  squad_bank_code    VARCHAR(10),
  role              VARCHAR(20) NOT NULL DEFAULT 'merchant',
                  -- ENUM: super_admin | admin | merchant
  hub_connected      BOOLEAN DEFAULT false,
  created_at         TIMESTAMPTZ DEFAULT now(),
  last_login_at      TIMESTAMPTZ
);

-- Bridge connection keys
CREATE TABLE hub_connection_tokens (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  token            VARCHAR(100) UNIQUE NOT NULL,  -- vph_conn_{64 hex chars}
  merchant_id       UUID NOT NULL REFERENCES merchants(id) ON DELETE CASCADE,
  created_at        TIMESTAMPTZ DEFAULT now(),
  expires_at        TIMESTAMPTZ NOT NULL,         -- created_at + 15 minutes
  used              BOOLEAN DEFAULT false,
  used_at           TIMESTAMPTZ
);

-- Connected native app devices
CREATE TABLE hub_devices (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  merchant_id       UUID NOT NULL REFERENCES merchants(id) ON DELETE CASCADE,
  device_name       VARCHAR(255),
  android_version   VARCHAR(20),
  app_version       VARCHAR(20),
  android_id        VARCHAR(100),                -- Android ANDROID_ID
  connected_at      TIMESTAMPTZ DEFAULT now(),
  last_seen_at      TIMESTAMPTZ DEFAULT now(),
  is_active         BOOLEAN DEFAULT true
);

-- JWT blacklist (for logout)
CREATE TABLE token_blacklist (
  jti              VARCHAR(100) PRIMARY KEY,      -- JWT ID claim
  expires_at       TIMESTAMPTZ NOT NULL          -- for cleanup
);

-- Indexes
CREATE INDEX idx_tokens_merchant ON hub_connection_tokens(merchant_id);
CREATE INDEX idx_tokens_expires ON hub_connection_tokens(expires_at);
CREATE INDEX idx_devices_merchant ON hub_devices(merchant_id);
CREATE INDEX idx_blacklist_exp ON token_blacklist(expires_at);

-- Auto-cleanup of expired tokens (run as cron or pg_cron)

```

```
DELETE FROM hub_connection_tokens WHERE expires_at < now() AND used = false;  
DELETE FROM token_blacklist WHERE expires_at < now();
```

06 SUPER ADMIN SEED — IMPLEMENTATION & CREDENTIALS

Super Admin Seed

The Super Admin account is seeded directly into the database via a migration script. It bypasses the Squad key authentication flow entirely — it uses a hardcoded credential set that the developer controls. This account has SUPER_ADMIN role and full unrestricted access to everything in the Hub. It is used for initial testing, onboarding merchants manually, and system administration.

SECURITY NOTE: The Super Admin credentials below are for development and hackathon demo only. Before any production deployment, rotate the password, change the email, and remove the hardcoded Squad key from the seed file. The seeded account must never use a real merchant's Squad key.

Seed Credentials

Field	Value
Email	admin@veriphy.ng
Password (for UI login)	Veriphy@Admin2024!
Role	super_admin
Business Name	Veri-phy System Admin
Squad Merchant ID	vph_system_superadmin_001
Squad Key (mock)	sk_test_superadmin_veriphy_internal_key_000
Hub Connected	true (pre-seeded as connected)
Dashboard URL	app.veriphy.ng → logs in automatically with seeded session

Seed Script

```
# scripts/seed_superadmin.py
# Run once: python scripts/seed_superadmin.py
# Idempotent – safe to run multiple times

import asyncio
from app.db.base import async_session
from app.db.models import Merchant
from app.core.security import encrypt_squad_key, mask_squad_key
from sqlalchemy import select

SUPER_ADMIN = {
    'squad_merchant_id': 'vph_system_superadmin_001',
    'business_name': 'Veri-phy System Admin',
    'email': 'admin@veriphy.ng',
    'squad_key_raw': 'sk_test_superadmin_veriphy_internal_key_000',
    'role': 'super_admin',
    'hub_connected': True,
}

async def seed():
```

```

async with async_session() as db:
    # Check if already seeded
    existing = await db.execute(
        select(Merchant).where(
            Merchant.squad_merchant_id == SUPER_ADMIN['squad_merchant_id']
        )
    )
    if existing.scalar_one_or_none():
        print('Super admin already seeded. Skipping.')
        return

    admin = Merchant(
        squad_merchant_id = SUPER_ADMIN['squad_merchant_id'],
        business_name     = SUPER_ADMIN['business_name'],
        email             = SUPER_ADMIN['email'],
        squad_key_encrypted = encrypt_squad_key(SUPER_ADMIN['squad_key_raw']),
        squad_key_hint     = mask_squad_key(SUPER_ADMIN['squad_key_raw']),
        role              = SUPER_ADMIN['role'],
        hub_connected     = SUPER_ADMIN['hub_connected'],
    )
    db.add(admin)
    await db.commit()
    print(f'Super admin seeded: {SUPER_ADMIN["email"]}')
    print(f'Role: {SUPER_ADMIN["role"]}')
    print('Access the hub at: app.veriphy.ng')
    print('Use Squad key: sk_test_superadmin_veriphy_internal_key_000')

asyncio.run(seed())

```

Super Admin Hub Login — Special Case

The Super Admin logs into the Hub using the /auth screen like any merchant. But since their Squad key is a mock key (not a real Squad key), the backend must short-circuit Squad validation for this account specifically.

```

# In app/services/auth.py – validate_squad_key()

INTERNAL_KEYS = {
    'sk_test_superadmin_veriphy_internal_key_000': {
        'merchant_id': 'vph_system_superadmin_001',
        'business_name': 'Veri-phy System Admin',
        'email': 'admin@veriphy.ng',
    }
}

async def validate_squad_key(key: str) -> SquadMerchantProfile:
    # Short-circuit for internal/seeded keys
    if key in INTERNAL_KEYS:
        return SquadMerchantProfile(**INTERNAL_KEYS[key])

    # Normal flow – call real Squad API
    async with httpx.AsyncClient() as client:
        resp = await client.get(
            'https://api.squadco.com/merchant/me',
            headers={'Authorization': f'Bearer {key}'},
            timeout=8.0
        )
    if resp.status_code == 401:
        raise InvalidSquadKeyError()
    resp.raise_for_status()
    return SquadMerchantProfile(**resp.json()['data'])

# This means: entering sk_test_superadmin_veriphy_internal_key_000

```

```
# on the /auth screen logs you in as Super Admin immediately,  
# with no Squad network call required.
```

What Super Admin Sees — Extra Routes

Super Admin sees everything a Merchant sees, plus these additional sidebar items that are hidden for the merchant role:

- ▶ All Merchants — directory of every merchant account on the platform
- ▶ System Health — Celery queue depth, FastAPI latency, Redis memory, uptime
- ▶ Model Registry — AI model versions, eval metrics, promote canary to prod
- ▶ Audit Log — system-wide (not scoped to one merchant)
- ▶ Role Management — promote/demote merchant accounts

Access Test: After running `seed_superadmin.py`, open `app.veriphy.ng`, enter `sk_test_superadmin_veriphy_internal_key_000` on the auth screen, click Continue. You should land on `/overview` with Super Admin role. The sidebar should show all admin-only routes. The top-left should show 'Veri-phy System Admin' with a green connected dot.

07 API ENDPOINT REFERENCE — AUTH & CONNECTION

API Endpoint Reference

Method	Route	Auth Required	Description
POST	/api/v1/auth/squad/connect	None	Validates Squad key, upserts merchant, issues JWT. The only unprotected write endpoint.
DELETE	/api/v1/auth/session	Bearer JWT	Logout — adds JWT to blacklist, returns 204.
GET	/api/v1/auth/me	Bearer JWT	Returns current session: merchant_id, business_name, role, hub_connected.
POST	/api/v1/connection/generate-key	Bearer JWT (merchant)	Generates a hub_connection_token for the authenticated merchant. Returns token + expires_at + qr_data.
GET	/api/v1/connection/status	Bearer JWT (merchant)	Polls Redis for connection status of the most recent token. Returns { status: waiting connected expired, device? }.
POST	/api/v1/connection/establish	Bearer JWT (native app)	Sent by native app with hub_connection_key + device_info. Validates token, links device, publishes to Redis.
GET	/api/v1/connection/device	Bearer JWT (merchant)	Returns connected device info or null if not connected.
DELETE	/api/v1/connection/device	Bearer JWT (merchant)	Disconnects the native app — marks device inactive, publishes disconnect event.
WS	/ws/live/{merchant_id}	JWT as ?token=param	Live WebSocket. Native app pushes verification and receipt events. Hub subscribes and updates.

08 DEVELOPER IMPLEMENTATION CHECKLIST

Developer Implementation Checklist

Complete these items in order. Do not proceed to the next item until the current one is verifiable.

Backend (veriphy-backend/)

1. Create merchants table with exact schema from Section 05
2. Create hub_connection_tokens table
3. Create hub_devices table
4. Create token_blacklist table

5. Create Alembic migration for all four tables
6. Implement `encrypt_squad_key()` and `mask_squad_key()` in `core/security.py`
7. Implement `validate_squad_key()` with `INTERNAL_KEYS` short-circuit
8. Implement `POST /api/v1/auth/squad/connect` with full upsert logic
9. Implement JWT issuance with RS256, `sub=merchant.id`, role claim
10. Implement JWT verification middleware for all protected routes
11. Implement role guard decorator `@require_role(...)`
12. Implement `merchant_id` scoping on all data queries
13. Implement `DELETE /api/v1/auth/session` with Redis blacklist
14. Implement `POST /api/v1/connection/generate-key`
15. Implement `GET /api/v1/connection/status` with Redis polling
16. Implement `POST /api/v1/connection/establish` with full validation sequence
17. Implement WebSocket `/ws/live/{merchant_id}` with JWT auth
18. Run `seed_superuser.py` and verify login with internal key
19. Write pytest tests for all auth endpoints — including invalid key, expired token, mismatched merchant cases

Frontend (apps/intelligence-hub/)

20. Implement `/auth` page with Squad key input, reveal toggle, continue button
21. Implement inline error states (invalid key, Squad unreachable)
22. Configure NextAuth with Credentials provider calling `/auth/squad/connect`
23. Implement `middleware.ts` protecting all routes except `/auth`
24. Implement Connection Indicator component with all 5 states
25. Implement Connection Modal with 3 internal states (generating, waiting, connected)
26. Implement 3-second polling via `useConnectionPoller` hook
27. Implement countdown timer from `expires_at`
28. Implement QR code display (use `qrcode.react` package)
29. Implement Copy Key button with clipboard API + copied confirmation
30. Test: open two browser tabs — one Hub session, simulate connection from second tab or Postman
31. Test: let key expire — verify expired state renders correctly
32. Test: login as Super Admin with internal key — verify admin routes visible
33. Test: login as merchant — verify admin routes NOT visible
34. Test: logout — verify redirect to `/auth`, JWT blacklisted

VERI-PHY · AUTH IMPLEMENTATION PROMPT · v1.0

Three auth layers. One spine. One merchant_id.